```
import pandas as pd
```

```
df = pd.read_csv('/content/Warehouse_and_Retail_Sales (1).csv')
```

```
df.head()
```

	YEAR	MONTH	SUPPLIER	ITEM CODE	ITEM DESCRIPTION	ITEM TYPE	RETAIL SALES	RETAIL TRANSFERS	WAREHOUSE SALES
0	2020	1	REPUBLIC NATIONAL DISTRIBUTING CO	100009	BOOTLEG RED - 750ML	WINE	0.00	0.0	2.0
1	2020	1	PWSWN INC	100024	MOMENT DE PLAISIR - 750ML	WINE	0.00	1.0	4.0
2	2020	1	RELIABLE CHURCHILL LLLP	1001	S SMITH ORGANIC PEAR CIDER - 18.7OZ	BEER	0.00	0.0	1.0
3	2020	1	LANTERNA DISTRIBUTORS INC	100145	SCHLINK HAUS KABINETT - 750ML	WINE	0.00	0.0	1.0
4	2020	1	DIONYSOS IMPORTS INC	100293	SANTORINI GAVALA WHITE - 750ML	WINE	0.82	0.0	0.0

Double-click (or enter) to edit

1.EXPLORATORY DATA ANALYSIS

```
df.head()
```

	YEAR	MONTH	SUPPLIER	ITEM CODE	ITEM DESCRIPTION	ITEM TYPE	RETAIL SALES	RETAIL TRANSFERS	WAREHOUSE SALES
0	2020	1	REPUBLIC NATIONAL DISTRIBUTING CO	100009	BOOTLEG RED - 750ML	WINE	0.00	0.0	2.0
1	2020	1	PWSWN INC	100024	MOMENT DE PLAISIR - 750ML	WINE	0.00	1.0	4.0
2	2020	1	RELIABLE CHURCHILL LLLP	1001	S SMITH ORGANIC PEAR CIDER - 18.7OZ	BEER	0.00	0.0	1.0
3	2020	1	LANTERNA DISTRIBUTORS INC	100145	SCHLINK HAUS KABINETT - 750ML	WINE	0.00	0.0	1.0
4	2020	1	DIONYSOS IMPORTS INC	100293	SANTORINI GAVALA WHITE - 750ML	WINE	0.82	0.0	0.0

```
df.tail()
```

	YEAR	MONTH	SUPPLIER	ITEM CODE	ITEM DESCRIPTION	ITEM TYPE	RETAIL SALES	RETAIL TRANSFERS	WARE
307640	2020	9	LEGENDS LTD	99753	DUTCHESS DE BOURGOGNE NR - 750ML	BEER	0.00	0.0	
307641	2020	9	ANHEUSER BUSCH INC	9997	HOEGAARDEN 4/6NR - 12OZ	BEER	66.12	37.0	
307642	2020	9	COASTAL BREWING COMPANY LLC	99970	DOMINION OAK BARREL STOUT 4/6 NR - 12OZ	BEER	2.25	0.0	
307643	2020	9	BOSTON BEER CORPORATION	99990	SAM ADAMS SUMMER VARIETY 12PK NR	BEER	20.50	0.0	
307644	2020	9	NaN	WC	WINE CREDIT	REF	0.00	0.0	

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 307645 entries, 0 to 307644
```

```
Data columns (total 9 columns):
 #   Column            Non-Null Count   Dtype
---  ------            --------------   -----
 0   YEAR              307645 non-null  int64
 1   MONTH             307645 non-null  int64
 2   SUPPLIER          307478 non-null  object
 3   ITEM CODE         307645 non-null  object
 4   ITEM DESCRIPTION  307645 non-null  object
 5   ITEM TYPE         307644 non-null  object
 6   RETAIL SALES      307642 non-null  float64
 7   RETAIL TRANSFERS  307645 non-null  float64
 8   WAREHOUSE SALES   307645 non-null  float64
dtypes: float64(3), int64(2), object(4)
memory usage: 21.1+ MB
```

```
df.isna().sum()
```

	0
YEAR	0
MONTH	0
SUPPLIER	167
ITEM CODE	0
ITEM DESCRIPTION	0
ITEM TYPE	1
RETAIL SALES	3
RETAIL TRANSFERS	0
WAREHOUSE SALES	0

dtype: int64

we have missing values as follows

supplier is missing 167

item type 1

retail sales 3

```
df.describe()
```

	YEAR	MONTH	RETAIL SALES	RETAIL TRANSFERS	WAREHOUSE SALES
count	307645.000000	307645.000000	307642.000000	307645.000000	307645.000000
mean	2018.438525	6.423862	7.024071	6.936465	25.294597
std	1.083061	3.461812	30.986238	30.237195	249.916798
min	2017.000000	1.000000	-6.490000	-38.490000	-7800.000000
25%	2017.000000	3.000000	0.000000	0.000000	0.000000
50%	2019.000000	7.000000	0.320000	0.000000	1.000000
75%	2019.000000	9.000000	3.267500	3.000000	5.000000
max	2020.000000	12.000000	2739.000000	1990.830000	18317.000000

```
df.shape
```

```
(307645, 9)
```

CHECING ON ERRORS ON THE DATA SET AND CORRECTING

```
Start coding or generate with AI.
```

```
#checking for duplicates
duplicates = df.duplicated()

print(duplicates)
```

```
0         False
1         False
2         False
3         False
4         False
          ...
307640    False
307641    False
307642    False
307643    False
307644    False
Length: 307645, dtype: bool
```

CHANGING DATA TYPE FROM OBJECT TO FLOAT

```
df['ITEM CODE'] = pd.to_numeric(df['ITEM CODE'], errors='coerce')
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 307645 entries, 0 to 307644
Data columns (total 9 columns):
```

```
 #   Column            Non-Null Count   Dtype
---  ------            --------------   -----
 0   YEAR              307645 non-null  int64
 1   MONTH             307645 non-null  int64
 2   SUPPLIER          307478 non-null  object
 3   ITEM CODE         307591 non-null  float64
 4   ITEM DESCRIPTION  307645 non-null  object
 5   ITEM TYPE         307644 non-null  object
 6   RETAIL SALES      307642 non-null  float64
 7   RETAIL TRANSFERS  307645 non-null  float64
 8   WAREHOUSE SALES   307645 non-null  float64
dtypes: float64(4), int64(2), object(3)
memory usage: 21.1+ MB
```

```
df.isna().sum()
```

	0
YEAR	0
MONTH	0
SUPPLIER	167
ITEM CODE	54
ITEM DESCRIPTION	0
ITEM TYPE	1
RETAIL SALES	3
RETAIL TRANSFERS	0
WAREHOUSE SALES	0

dtype: int64

CHECKING IF THERE IS ANY DUPLICATES

```
# checking for duplicates
print(f"\nNumber of duplicate rows: {df.duplicated().sum()}")
```

```
Number of duplicate rows: 0
```

Double-click (or enter) to edit

HANDLING MISSING VALUES

```
df['RETAIL SALES'].unique()
```

```
array([0.0000e+00, 8.2000e-01, 2.7600e+00, ..., 1.7916e+02, 3.8192e+02,
       1.4450e+03])
```

```
df['ITEM TYPE'].unique()
```

```
array(['WINE', 'BEER', 'LIQUOR', 'STR_SUPPLIES', 'KEGS', 'REF', 'DUNNAGE',
       'NON-ALCOHOL', nan], dtype=object)
```

```
df['ITEM CODE'].unique()
```

```
array([100009., 100024.,   1001., ...,  11203.,  10071.,  10090.])
```

```
df['SUPPLIER'].unique()
```

```
       'CASA CHIRICO LLC', 'BLACK ANKLE VINEYARDS LLC',
       'DOGFISH HEAD CRAFT BREWERY LLC',
       'NIEBAUM COPPOLA ESTATE WINERY LP', 'DISTRICT CIDER COMPANY INC',
       'ONE EIGHT DISTILLING LLC',
       'RED MOUNTAIN DISTILLING AND SPIRITS LLC',
       'CHARM CITY BEVERAGE LLC', 'CHEERS DISTRIBUTING INC',
       'THE DOG BEVERAGE CO INC', 'MARKERS EDGE LLC', 'FIOR DI SOLE LLC',
       'SNR HOLDINGS LLC', 'RSL HOLDINGS INC', 'GJS SALES INC',
       'TOBACCO BARN DISTILLERY', 'WITH MALUS AFORETHOUGHT LLC'],
      dtype=object)
```

```
# for categorical cols
categorical_cols = df.select_dtypes(include='object', exclude='number').columns

for col in categorical_cols:
  mode_value = df[col].mode()[0]
  df[col].fillna(mode_value, inplace=True)

df.isna().sum()
```

```
/tmp/ipykernel_3514/2548598655.py:6: FutureWarning: A value is trying to be set on a cop
The behavior will change in pandas 3.0. This inplace method will never work because the

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col


  df[col].fillna(mode_value, inplace=True)
```

	0
YEAR	0
MONTH	0
SUPPLIER	0
ITEM CODE	54
ITEM DESCRIPTION	0
ITEM TYPE	0
RETAIL SALES	3
RETAIL TRANSFERS	0
WAREHOUSE SALES	0

dtype: int64

```
#missing value for retail sale
df['RETAIL SALES'].fillna(df['RETAIL SALES'].mean(), inplace=True)
```

```
/tmp/ipykernel_3514/741853347.py:2: FutureWarning: A value is trying to be set on a copy
The behavior will change in pandas 3.0. This inplace method will never work because the

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col
```

```
df['RETAIL SALES'].fillna(df['RETAIL SALES'].mean(), inplace=True)
```

```
df.isna().sum()
```

	0
YEAR	0
MONTH	0
SUPPLIER	0
ITEM CODE	54
ITEM DESCRIPTION	0
ITEM TYPE	0
RETAIL SALES	0
RETAIL TRANSFERS	0
WAREHOUSE SALES	0

dtype: int64

```
#handling missing values fo item code
df['ITEM CODE'].fillna(df['ITEM CODE'].mean(), inplace=True)
df.isna().sum().any()
```

```
/tmp/ipykernel_3514/2427594918.py:2: FutureWarning: A value is trying to be set on a cop
The behavior will change in pandas 3.0. This inplace method will never work because the

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col


  df['ITEM CODE'].fillna(df['ITEM CODE'].mean(), inplace=True)
np.False_
```

```
df.isna().sum()
```

	0
YEAR	0
MONTH	0
SUPPLIER	0
ITEM CODE	0
ITEM DESCRIPTION	0
ITEM TYPE	0
RETAIL SALES	0
RETAIL TRANSFERS	0
WAREHOUSE SALES	0

dtype: int64

```
df.describe()
```

	YEAR	MONTH	ITEM CODE	RETAIL SALES	RETAIL TRANSFERS	WAREH S
count	307645.000000	307645.000000	3.076450e+05	307645.000000	307645.000000	307645.00
mean	2018.438525	6.423862	1.641144e+05	7.024071	6.936465	25.29
std	1.083061	3.461812	1.361029e+05	30.986087	30.237195	249.91
min	2017.000000	1.000000	2.000000e+00	-6.490000	-38.490000	-7800.00
25%	2017.000000	3.000000	4.810600e+04	0.000000	0.000000	0.00
50%	2019.000000	7.000000	8.402600e+04	0.320000	0.000000	1.00
75%	2019.000000	9.000000	3.180820e+05	3.270000	3.000000	5.00
max	2020.000000	12.000000	3.480003e+06	2739.000000	1990.830000	18317.00

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 307645 entries, 0 to 307644
Data columns (total 9 columns):
 #   Column            Non-Null Count   Dtype
---  ------            --------------   -----
 0   YEAR              307645 non-null  int64
 1   MONTH             307645 non-null  int64
 2   SUPPLIER          307645 non-null  object
 3   ITEM CODE         307645 non-null  float64
 4   ITEM DESCRIPTION  307645 non-null  object
 5   ITEM TYPE         307645 non-null  object
 6   RETAIL SALES      307645 non-null  float64
 7   RETAIL TRANSFERS  307645 non-null  float64
 8   WAREHOUSE SALES   307645 non-null  float64
```

```
dtypes: float64(4), int64(2), object(3)
memory usage: 21.1+ MB
```

3.EXPLORATORY DATA ANALYSIS

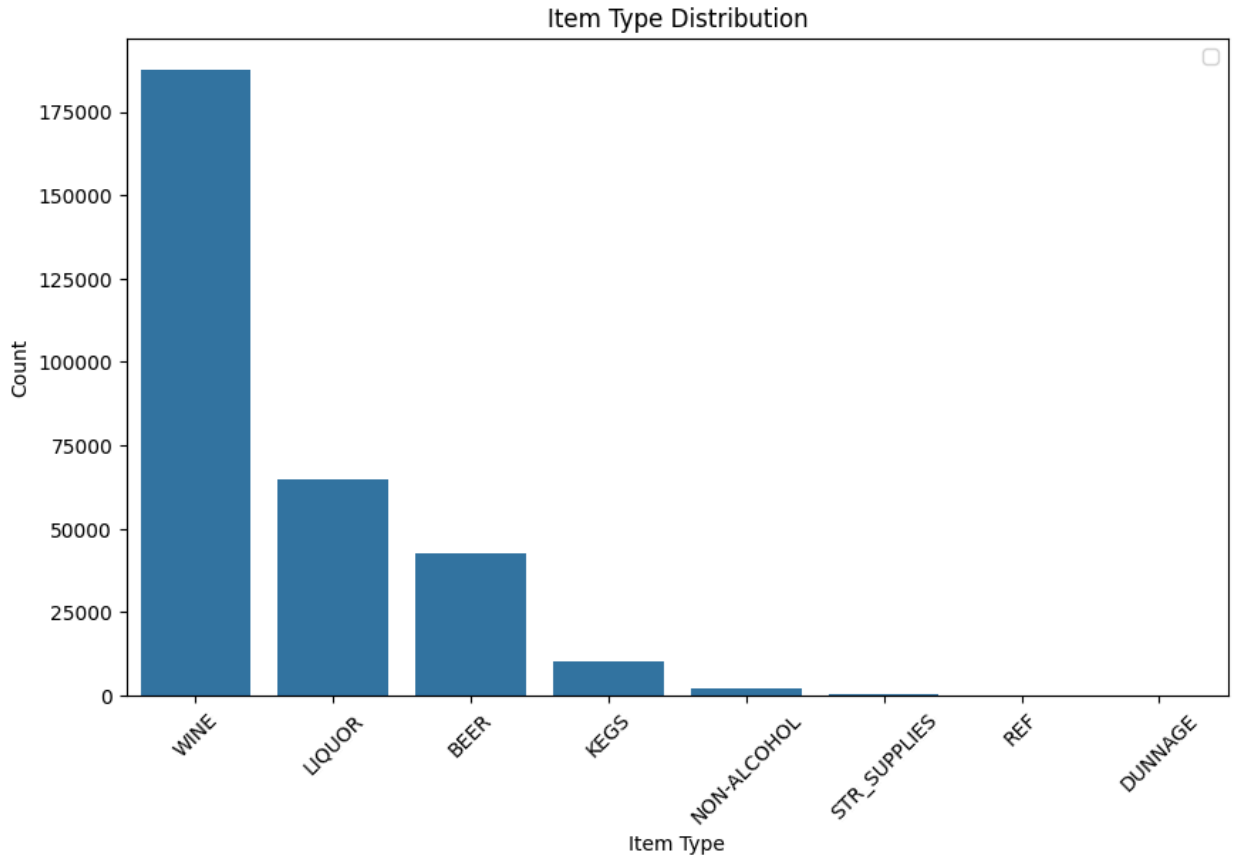
```
#CHECKING FOR ITEM TYPE DISTRIBUTION
item_type_counts = df['ITEM TYPE'].value_counts()
print(item_type_counts)
```

```
ITEM TYPE
WINE            187641
LIQUOR           64910
BEER             42413
KEGS             10146
NON-ALCOHOL       1908
STR_SUPPLIES       405
REF                127
DUNNAGE             95
Name: count, dtype: int64
```

```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
```

```
#bar graph for item type
plt.figure(figsize=(10, 6))
sns.barplot(x=item_type_counts.index, y=item_type_counts.values)
plt.xlabel('Item Type')
plt.ylabel('Count')
plt.title('Item Type Distribution')
plt.xticks(rotation=45)
plt.legend()
plt.show()
```

```
/tmp/ipykernel_3514/2944142862.py:8: UserWarning: No artists with labels found to put in
  plt.legend()
```



- wine was the most sold item through out the years while Dunnage was the least sold

```
#bar graph for item type by warehouse sale
item_type_warehouse_sales = df.groupby('ITEM TYPE')['WAREHOUSE SALES'].sum()
print(item_type_warehouse_sales)
```

```
ITEM TYPE
BEER            6527236.51
DUNNAGE         -121454.00
KEGS             118431.00
LIQUOR            94906.27
NON-ALCOHOL       26149.59
REF              -20499.00
STR_SUPPLIES          0.00
WINE            1156985.91
Name: WAREHOUSE SALES, dtype: float64
```

```
item_types_retailsales = df.groupby('ITEM TYPE')['RETAIL SALES'].sum()
print(item_types_retailsales)
```

```
ITEM TYPE
BEER            574220.530000
DUNNAGE              0.000000
KEGS                 0.000000
```

```
LIQUOR          802691.430000
NON-ALCOHOL      34105.382214
REF                663.630000
STR_SUPPLIES      2740.880000
WINE            746498.590000
Name: RETAIL SALES, dtype: float64
```

```
item_type_retail_transferred = df.groupby('ITEM TYPE')['RETAIL TRANSFERS'].sum()
print(item_type_retail_transferred)
```

```
ITEM TYPE
BEER            566714.00
DUNNAGE              0.00
KEGS                -1.00
LIQUOR          794735.71
NON-ALCOHOL      26666.38
REF                388.92
STR_SUPPLIES     10846.58
WINE            734618.04
Name: RETAIL TRANSFERS, dtype: float64
```

```
#line graph for retail sale per year
df['YEAR'] = pd.to_datetime(df['YEAR'], format='%Y')
yearly_retail_sales = df.groupby(df['YEAR'].dt.year)['RETAIL SALES'].sum()
print(yearly_retail_sales)
```

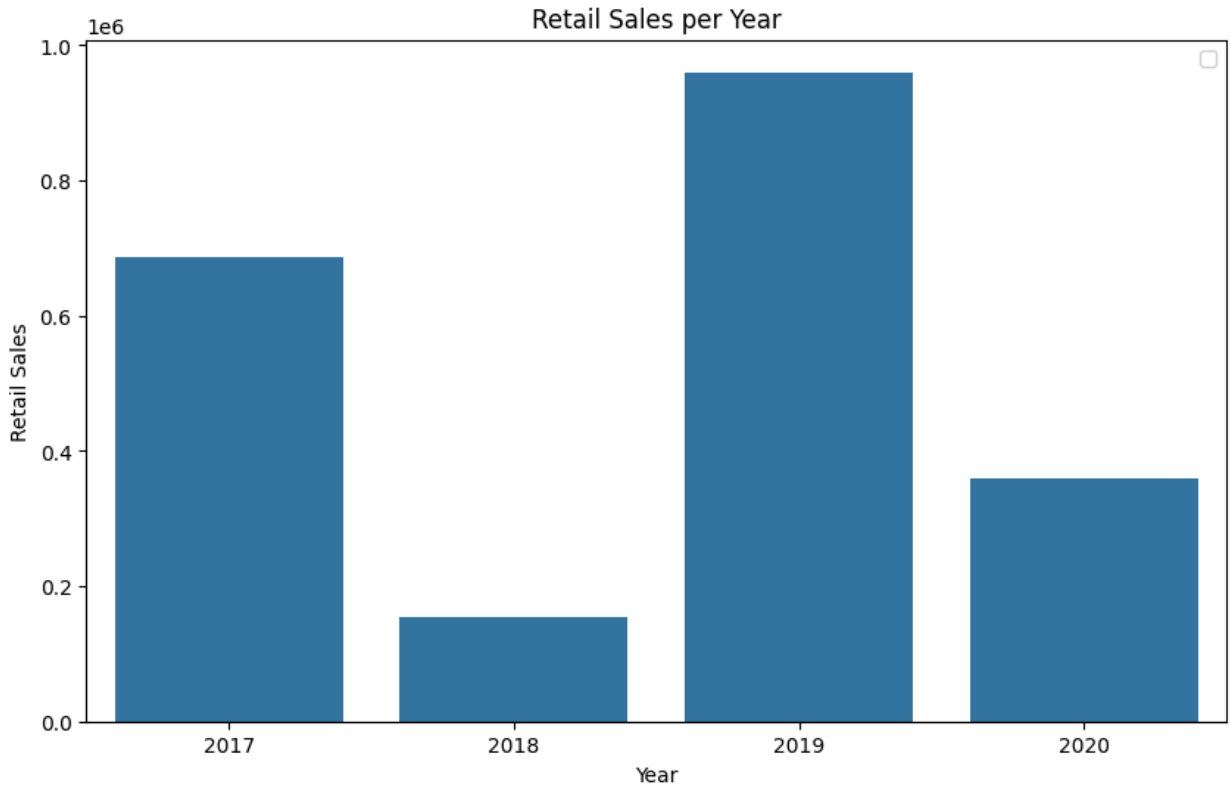
```
YEAR
2017    686734.570000
2018    153595.900000
2019    960191.200000
2020    360398.772214
Name: RETAIL SALES, dtype: float64
```

```
#bar graph for retail sale per year
plt.figure(figsize=(10, 6))
sns.barplot(x=yearly_retail_sales.index, y=yearly_retail_sales.values)
plt.title('Retail Sales per Year')
plt.xlabel('Year')
plt.ylabel('Retail Sales')
plt.legend()
plt.show
```

```
/tmp/ipykernel_3514/3771049761.py:7: UserWarning: No artists with labels found to put in
  plt.legend()
```

```
matplotlib.pyplot.show
def show(*args, **kwargs) -> None
```

```
/usr/local/lib/python3.12/dist-packages/matplotlib/pyplot.py
Display all open figures.

Parameters
----------
block : bool, optional
```



- The year 2019 had the highest retail sales followed by 2017
- the year 2018 had the lowest retail sales

Start coding or generate with AI.

```
#bar graph for warehouse sales per year
yearly_warehouse_sales = df.groupby(df['YEAR'].dt.year)['WAREHOUSE SALES'].sum()
print(yearly_warehouse_sales)
```

```
YEAR
2017    2333849.13
2018     519526.19
2019    3543371.23
2020    1385009.73
Name: WAREHOUSE SALES, dtype: float64
```

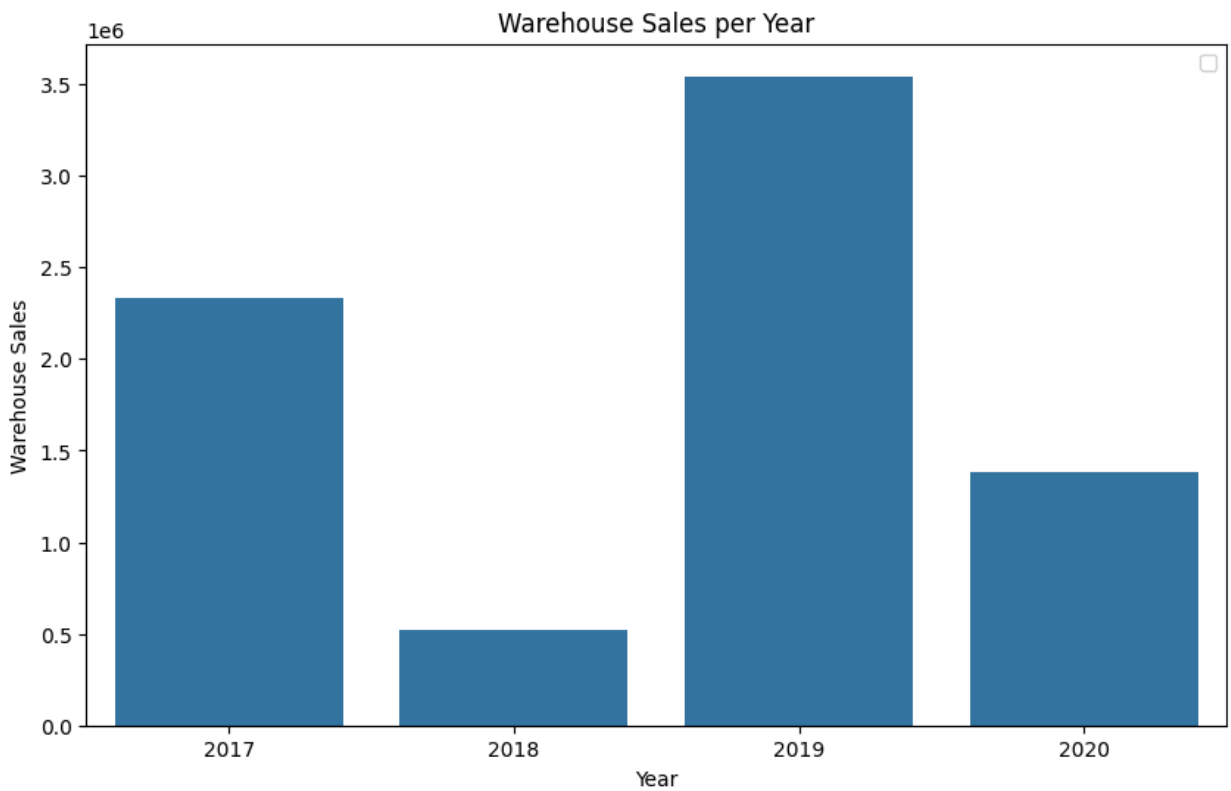
```
plt.figure(figsize=(10, 6))
sns.barplot(x=yearly_warehouse_sales.index, y=yearly_warehouse_sales.values)
plt.title('Warehouse Sales per Year')
plt.xlabel('Year')
plt.ylabel('Warehouse Sales')
plt.legend()
plt.show
```

```
/tmp/ipykernel_3514/3634621292.py:6: UserWarning: No artists with labels found to put in
  plt.legend()
```

matplotlib.pyplot.show
def show(*args, **kwargs) -> None

/usr/local/lib/python3.12/dist-packages/matplotlib/pyplot.py
Display all open figures.

Parameters
---
block : bool, optional



- The year 2019 had the highest warehouse sale followed by 2017
- The year 2018 had the lowest ware house sale

Start coding or generate with AI.

```
#bar graph for retail transferred per year
yearly_retail_transferred = df.groupby(df['YEAR'].dt.year)['RETAIL TRANSFERS'].sum()
```

```
print(yearly_retail_transferred)
```

```
YEAR
2017    676620.50
2018    153652.92
2019    957562.40
2020    346132.81
Name: RETAIL TRANSFERS, dtype: float64
```

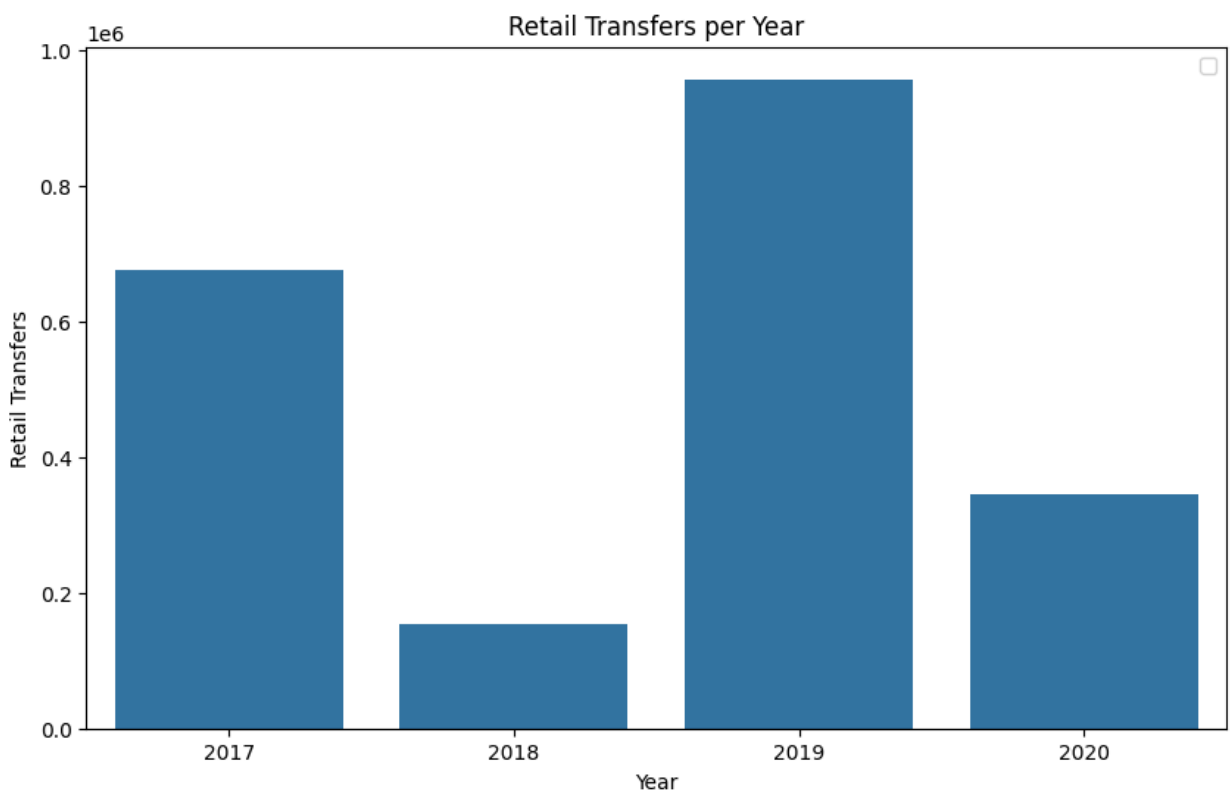
```
#RETAIL TRANSFERRED PER YEAR
plt.figure(figsize=(10, 6))
sns.barplot(x=yearly_retail_transferred.index, y=yearly_retail_transferred.values)
plt.title('Retail Transfers per Year')
plt.xlabel('Year')
plt.ylabel('Retail Transfers')
plt.legend()
plt.show
```

```
/tmp/ipykernel_3514/411807859.py:7: UserWarning: No artists with labels found to put in
  plt.legend()
```

matplotlib.pyplot.show
def show(*args, **kwargs) -> None

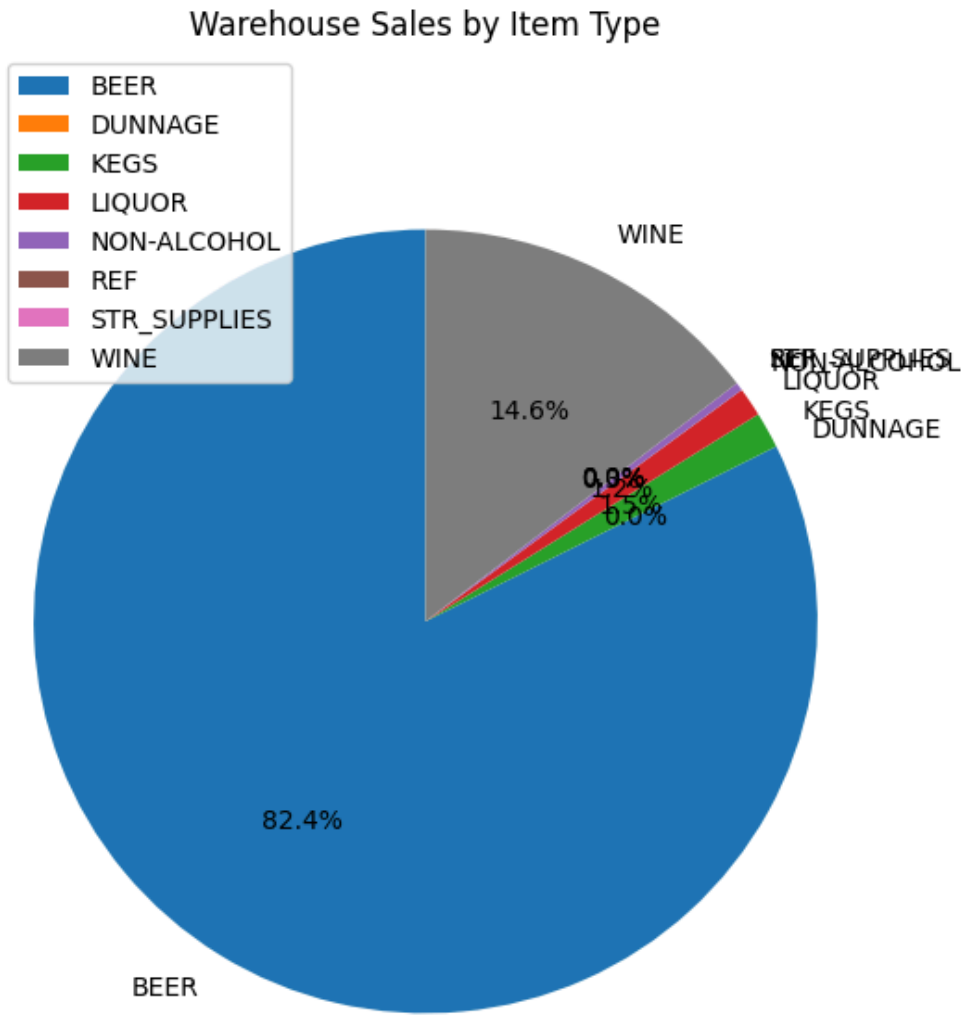
/usr/local/lib/python3.12/dist-packages/matplotlib/pyplot.py
Display all open figures.

Parameters
---
block : bool, optional



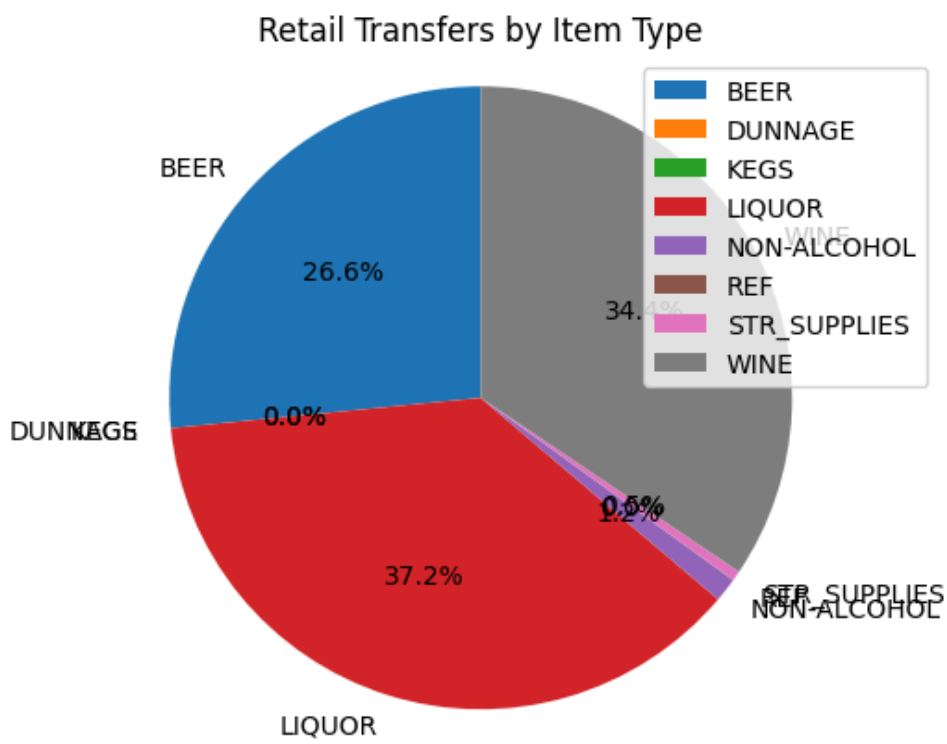
- 2019 had the highest retail transferred followed by the year 2017
- the year 2018 had the lowest retail transferred

```
#pie chart
# Ensure warehouse sales are non-negative for the pie chart
df['WAREHOUSE SALES'] = df['WAREHOUSE SALES'].apply(lambda x: max(0, x))

item_type_warehouse_sales = df.groupby('ITEM TYPE')['WAREHOUSE SALES'].sum()

plt.figure(figsize=(6, 8))
plt.pie(item_type_warehouse_sales, labels=item_type_warehouse_sales.index, autopct='%1
plt.axis('equal')
plt.legend()
plt.title('Warehouse Sales by Item Type')
plt.show()
```



- in warehouse sales bear was the most sold item followed by wine

```
#pie chart for retail transferred
# making sure no negative for these set
df['RETAIL TRANSFERS'] = df['RETAIL TRANSFERS'].apply(lambda x: max(0, x))

item_type_retail_transferred = df.groupby('ITEM TYPE')['RETAIL TRANSFERS'].sum()
plt.figure.figsize=(6, 8)
plt.pie(item_type_retail_transferred, labels=item_type_retail_transferred.index, autop
plt.axis('equal')
plt.legend()
plt.title('Retail Transfers by Item Type')
plt.show()
```



- in ware house transferref liqour was the most transferred followed closely by wine